

Università degli Studi di Salerno

Corso di Ingegneria del Software

Tasky
Test Cases
Versione 0.7



Data: 19/12/2025

Progetto: Tasky	Versione: 0.7
Documento: Test Cases	Data: 19/12/2025

Coordinatore del progetto:

Nome	Matricola
Luca Vincenzo Lambiase Fontana	0512113338

Partecipanti:

Nome	Matricola
Ilaria Lastra	0512120280
Rinaldo Perna	0512119593

Scritto da:	Ilaria Lastra
--------------------	---------------

Revision History

Data	Versione	Descrizione	Autore
01/10/2025	0.1	Presentazione Proposta di Progetto	Lastra Ilaria
10/10/2025	0.2	Presentazione Problem Statement con descrizione degli scenari principali dell'applicazione, i suoi requisiti e gli ambienti target	Lastra Ilaria
20/10/2025	0.3	Presentazione del Requirements Analysis Document, con perfezionamento dei requisiti esposti nel Problem Statement e definizione dei criteri di successo e dello scopo e ambito del sistema	Lastra Ilaria
08/11/2025	0.3.1	Presentazione dell'Object Model e del Dynamic Model, con Class Diagram, Sequence Diagram e Statechart Diagram relativi al Requirements Analysis Document	Lastra Ilaria
19/11/2025	0.4	Presentazione del System Design Document, con discussione dei design goals, dell'architettura e mappatura hardware, software.	Lastra Ilaria, Luca Vincenzo Lambiase Fontana, Rinaldo Perna
19/12/2025	0.5	Presentazione dell'Object Design Document con discussione dei compromessi della progettazione ad oggetti, linee guida per la documentazione dell'interfaccia e packaging dei sottosistemi	Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna
19/12/2025	0.6	Presentazione del Test Plan Document, con descrizione delle feature da testare e non, dei criteri di superamento del test, approccio e materiali	Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna
19/12/2025	0.7	Presentazione del Test Cases Document, con descrizione dettagliata dei casi di test introdotti nel Test Plan Document	Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna

Sommario

<i>1. Test case specification identifier</i>	<i>6</i>
<i>2. Test items</i>	<i>6</i>
<i>3. Input specifications</i>	<i>6</i>
<i>4. Output specifications.....</i>	<i>8</i>
<i>5. Environmental needs</i>	<i>9</i>
<i>6. Special procedural requirements</i>	<i>10</i>
<i>7. Intercase dependencies</i>	<i>10</i>

1. Test case specification identifier

Il seguente documento è volto alla descrizione in dettaglio dei casi di test, progettati per verificare le funzionalità critiche del sistema Tasky. Ogni caso di test è mappato su uno specifico requisito funzionale (Use Case) e include i passaggi per l'esecuzione, i dati di input e i risultati attesi.

I nostri casi di test saranno identificati come segue:

- **TC_01 (Login):** Verifica accesso con credenziali valide e generazione token
- **TC_02 (Aggiunta Task):** Verifica corretto inserimento task tramite form con scadenza valida
- **TC_03 (Sospensione Task Manuale):** Verifica che un Manager possa cambiare lo stato di un task in "Sospeso"
- **TC_04 (Eliminazione Task):** Verifica che solo un Manager possa eliminare un task dal database
- **TC_05 (Visualizzazione Progetti):** Verifica filtraggio progetti per dipartimento del manager
- **TC_06 (Aggiunta Progetto):** Verifica corretto inserimento progetto tramite form
- **TC_07 (Eliminazione Progetto):** Verifica eliminazione da parte del manager e cascading sui task
- **TC_08 (Scheduler Automatico):** Verifica che i task scaduti vengano automaticamente impostati su "Sospeso" dal sistema.
- **TC_09 (Visibilità Colleghi):** Verifica che un dipendente possa visualizzare la lista dei colleghi del proprio dipartimento, ma non di altri.
- **TC_10 (Progetti Manager):** Verifica il recupero dei progetti gestiti da uno specifico manager

2. Test items

I componenti che dovranno essere oggetto dei nostri test sono:

- **User Management:** consiste nella componente riservata alla **gestione delle autenticazioni** utente. I casi di test che ricopriranno la seguente componente sono: **TC_01**
- **Task Management:** consiste nella componente addetta alla logica di **gestione delle task**. I casi di test che ricopriranno la seguente componente sono: **TC_02, TC_03, TC_04**
- **Project Management:** consiste nella componente addetta alla logica di **gestione dei progetti** e i casi di test coinvolti sono: **TC_05, TC_06, TC_07, TC_10**
- **Department Management:** consiste nella componente addetta alla **visualizzazione dei membri** di uno specifico dipartimento e il caso di test coinvolto è **TC_09**.

3. Input specifications

Nella seguente sezione andremo ad elencare i rispettivi input richiesti da ogni caso di test descritto all'interno del seguente documento:

- **ID: TC_01**
 - **Precondizioni:** L'utente (Manager o Dipendente) deve essere già registrato nel database.
 - **Input:** Email: dip_test@example.com
Password: pwd
 - **Passaggi:** Inviare una richiesta POST /api/login con le credenziali indicate di prova.

Verificare la risposta del server

- **ID: TC_02**
 - **Precondizioni:** Token manager valido e Progetto e Dipendente esistenti
 - **Input:** Token: <Manager_Token>
Dati Task: Nome, Descrizione, Data Inizio, Data Fine (futura), ID Progetto, Email Dipendente
 - **Passaggi:** Inviare POST /api/add/Task con il payload JSON
- **ID: TC_03**
 - **Precondizioni:** Esiste un task in stato “In corso”
 - **Input:** Token: <Manager_Token>
Id Task: <ID_Task>
Nuovo Stato : <Sospeso>
 - **Passaggi:** Inviare una richiesta POST /api/update/Task specificando il nuovo stato
- **ID: TC_04**
 - **Precondizioni:** Token Manager valido.
 - **Input:** Token = <Manager_Token>
 - Id Task: <ID_Task_Da_Eliminare>
 - **Passaggi:** Inviare POST /api/delete/Task. Provare a recuperare il task eliminato (opzionale).
- **ID: TC_05**
 - **Precondizioni:** Token Manager del Dipartimento A.
 - **Input:** Token = <Manager_Token_Dept_A>
ID Dipartimento: <ID_Dept_A>
 - **Passaggi:** Inviare una richiesta POST /api/project/by-department
- **ID: TC_06**
 - **Precondizioni:** Token manager valido.
 - **Input:** Nome = “Progetto Beta”
Budget: 5000.00
Date: Inizio/Fine valide
 - **Passaggi:** Inviare una richiesta POST /api/add/project
- **ID: TC_07**
 - **Precondizioni:** Un progetto con almeno un task associato
 - **Input:** Id Progetto: <ID_Progetto>
 - **Passaggi:** Inviare una richiesta POST /api/delete/Project
- **ID: TC_08**
 - **Precondizioni:** Un task con data_fine impostata a ieri e stato “In Corso”
 - **Input:** Id task: <ID_Task>
 - **Passaggi:** Invocare manualmente il trigger dello scheduler: POST /api/debug/scheduler/check-overdue e verificare lo stato del task
- **ID: TC_09**
 - **Precondizioni:** Dipendente del Dipartimento esempio 1
 - **Input:** Id Dipartimento: <ID_Dipartimento>
 - **Passaggi:** Richiedere la lista colleghi del Dipartimento 1 (POST /api/dipendenti/by-department) e prova della richiesta lista colleghi Dipartimento 2
- **ID: TC_10**
 - **Input:** Email = <Email_Manager>
 - **Passaggi:** Inviare una richiesta POST /api/projects/by-manager

4. Output specifications

Di seguito saranno riportati gli output attesi da ogni singolo caso di test elencato precedentemente:

- **TC_01**
 - **Output atteso:** Codice 200 OK e oggetto JSON contenente la stringa Bearer Token e i dati dell'utente
 - **Stato finale del sistema:** Creazione di una sessione attiva lato server
- **TC_02**
 - **Output atteso:** Codice 201 Created, messaggio "Task aggiunto con successo" e visualizzazione del task nella lista
 - **Stato finale del sistema:** Nuovo record inserito nella tabella "Task" con attributo Stato = "In Corso"
- **TC_03**
 - **Output atteso:** Codice 200 OK e visualizzazione dello stato "Sospeso" nella UI del task
 - **Stato finale del sistema:** Attributo Stato del task aggiornato a "Sospeso"; campo Motivazione popolato con il testo inserito.
- **TC_04**
 - **Output atteso:** Codice 204 No Content e rimozione istantanea del task dalla dashboard utente
 - **Stato finale del sistema:** Record rimosso permanentemente dalla tabella Task
- **TC_05**
 - **Output atteso:** Codice 200 OK e lista JSON di progetti filtrati dove ID_Dipartimento_Progetto == ID_Dipartimento_Manager
 - **Stato finale del sistema:** Nessuna variazione (operazione di sola lettura)
- **TC_06**
 - **Output atteso:** Codice 201 Created, messaggio di conferma e reindirizzamento alla vista progetto
 - **Stato finale del sistema:** Nuovo record inserito nella tabella Project con budget e dipartimento corretti
- **TC_07**
 - **Output atteso:** Codice 204 No Content, messaggio di successo per l'eliminazione a cascata
 - **Stato finale del sistema:** Rimozione del record Project e di tutti i record Task con FK_Project_ID corrispondente
- **TC_08**
 - **Output atteso:** Log di sistema: "Task ID impostato su Sospeso per scadenza". Notifica opzionale utente
 - **Stato finale del sistema:** Attributo Stato modificato da "In corso" a "Sospeso" per tutti i task con Data_Fine < Current_Date
- **TC_09**

- **Output atteso:** Codice 200 OK e lista utenti appartenenti esclusivamente allo stesso `Department_ID` dell'utente in sessione
 - **Stato finale del sistema:** Nessuna variazione
- **TC_10**
 - **Output atteso:** Codice 200 OK ed elenco completo dei progetti in cui l'utente loggato è indicato come Coordinatore/Manager
 - **Stato finale del sistema:** Nessuna variazione

5.Environmental needs

Per l'esecuzione dei casi di test sopra descritti, l'ambiente di prova deve soddisfare i seguenti requisiti minimi:

- **Piattaforma Hardware**
 - **Workstation del Tester:** PC con almeno 8GB di RAM e processore quad-core per l'esecuzione fluida degli strumenti di monitoraggio e dei browser
 - **Server di Staging:** Un'istanza server (locale o cloud) dedicata al testing, configurata per replicare l'ambiente di produzione, garantendo l'isolamento dei dati
- **Piattaforma Software**
 - **Sistema Operativo:** Ambiente basato su Unix/Linux (consigliato Ubuntu 22.04 LTS) o Windows con WSL2 per garantire la compatibilità con le librerie Python.
 - **Backend:** Interprete **Python 3.10+** con il framework **Flask** e le dipendenze elencate nel file `requirements.txt`.
 - **Database:** Istanza di **PostgreSQL** (versione 14 o superiore) con schema aggiornato secondo l'ultima migrazione di **SQLAlchemy**.
 - **Interfaccia Utente:** Browser moderni (Chrome v.110+, Firefox v.105+) con supporto per la crittografia TLS.
- **Strumenti di Test, Driver e Stub**
 - **Client API:** Utilizzo di **Postman** o **Insomnia** per l'invio delle richieste REST e la verifica dei Bearer Token.
 - **Framework di Testing:** Libreria **PyTest** per l'esecuzione degli Unit Test e degli Integration Test automatizzati.
 - **Driver Database:** Driver `psycopg2` per la comunicazione tra **SQLAlchemy** e **PostgreSQL**.
 - **Stub di Servizio:** In assenza di un server SMTP reale, deve essere utilizzato un **Mail-Stub** (come Mailhog) per intercettare eventuali notifiche di sistema inviate durante i test di gestione task.
 - **Mock Objects:** Utilizzo della libreria `unittest.mock` per simulare componenti non ancora integrati o servizi esterni di terze parti (es. sistemi di storage cloud).

6.Special procedural requirements

I vincoli e le azioni manuali indispensabili per la corretta esecuzione dei test case, garantendo che le condizioni al contorno non alterino i risultati sono:

6.1 Sequenza di Autenticazione (Prerequisito Universale)

Ad eccezione del test TC_01 (Login), tutti i test case richiedono che l'operatore ottenga preventivamente un **Bearer Token** valido.

- **Procedura:** Il token ottenuto dalla risposta del server deve essere inserito nell'header HTTP Authorization di ogni richiesta successiva. La scadenza del token è fissata a 60 minuti; se il test supera tale durata, la procedura di login deve essere ripetuta.

6.2 Simulazione del Role-Based Access Control (RBAC)

Per validare i requisiti di sicurezza (es. TC_03 e TC_07), è necessario alternare le sessioni tra due profili distinti:

1. **Profilo Manager:** Per eseguire operazioni di scrittura, sospensione ed eliminazione a cascata.
2. **Profilo Dipendente:** Per verificare il corretto sollevamento di eccezioni di tipo 403 Forbidden nel tentativo di eseguire operazioni privilegiate.

6.3 Gestione dello Stato e Pulizia dei Dati (Data Cleanup)

Per i test distruttivi (come TC_07 - Eliminazione Progetto), è obbligatorio seguire questa procedura:

- **Pre-condizione:** Eseguire uno script di *seeding* del database per ricreare l'alberatura "Dipartimento -> Progetto -> Task" prima di ogni iterazione.
- **Post-condizione:** Verificare manualmente o tramite query SQL l'effettiva assenza di record orfani nella tabella Task per confermare il successo della logica di *cascading delete*.

6.4 Intervento dell'Operatore e Validazione UI

Per i test che coinvolgono l'interfaccia web (Front-end), il tester deve:

- **Gestione Modal:** Intervenire manualmente per confermare i pop-up di avviso ("Sei sicuro di voler eliminare questo progetto?") prima che la chiamata API venga inoltrata al backend.
- **Ispezione Console:** Monitorare la console del browser per intercettare eventuali errori di parsing dei dati o violazioni della *Content Security Policy* (CSP).

6.5 Vincoli Temporalì e Logica di Scadenza

Per testare la funzionalità di "Sospensione Automatica per Scadenza Violata" (indicata nel RAD):

- **Procedura:** L'operatore deve modificare manualmente l'orologio di sistema del server di test o alterare il campo Data Fine di un task nel database a un timestamp passato, per poi innescare il servizio di controllo stati del sistema Tasky.

7.Intercase dependencies

I nostri casi d'uso sono caratterizzati da legami gerarchici e propedeutici. Per ottimizzare il processo, i test devono essere eseguiti seguendo l'ordine di dipendenza indicato, onde evitare falsi negativi dovuti all'assenza di oggetti nel sistema.

Caso di test	Dipendenze	Motivazioni e vincoli
TC_01	Nessuna	È il test radice. Fornisce il Bearer Token necessario per tutte le chiamate API protette

TC_02	TC_01 e TC_06	Richiede una sessione valida e l'esistenza di un Progetto a cui associare il task
TC_03	TC_01 e TC_02	Richiede che un task sia stato creato con successo e l'utente sia autenticato come Manager
TC_04	TC_01 e TC_02	Richiede l'esistenza del record Task nel database per verificarne la rimozione fisica.
TC_05	TC_01 e TC_06	Necessita di record nella tabella Project per validare l'algoritmo di filtraggio per dipartimento.
TC_06	TC_01	Richiede il ruolo Manager. Deve precedere i test che operano su collezioni di progetti.
TC_07	TC_01 e TC_02 e TC_06	Dipendenza Critica: Per testare il <i>cascading</i> , devono esistere sia il progetto che almeno un task associato.
TC_08	TC_02	Il sistema deve avere task con data scadenza < <i>current_date</i> creati nel TC_02 per attivare il trigger.
TC_09	TC_01	Richiede che il database sia popolato con utenti appartenenti a dipartimenti diversi.
TC_10	TC_01 e TC_06	Richiede che siano stati creati progetti specificamente assegnati al Manager in sessione.